

DOCUMENTATION WEEK 1

MONDAY

morning

- overall introduction to the colabor modules
- presentation of sound artist

afternoon

- introduction of the module
- short personal introduction
- exchange on coding knowledge / expectations
- introduction to groups

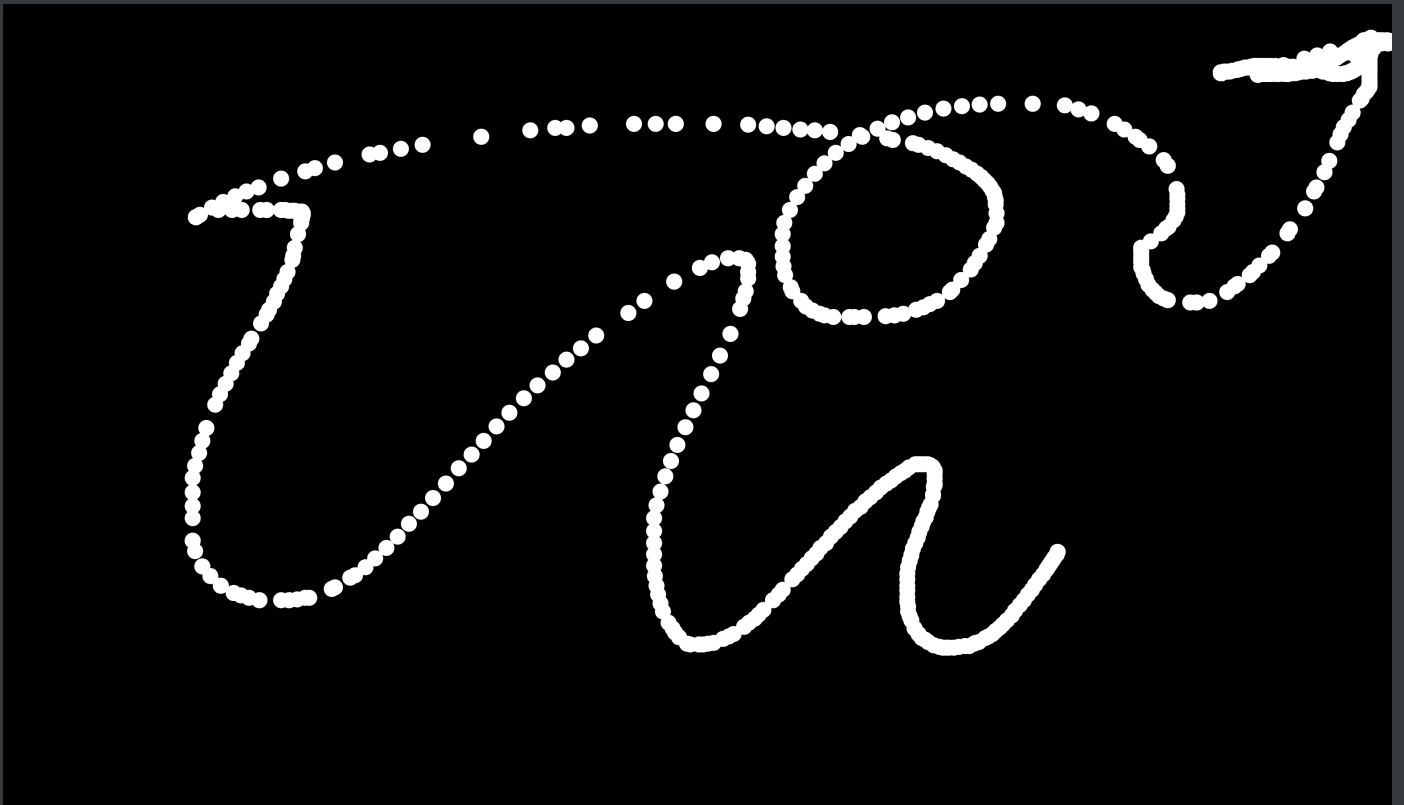
TUESDAY

morning

- introduction to p5.live

BASIC PAINT PROGRAM

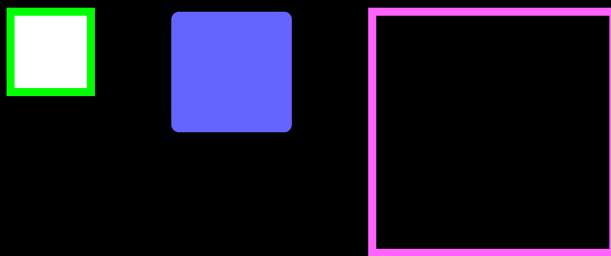
```
function setup() {  
  createCanvas(windowWidth, windowHeight);  
  frameRate(200)  
  background(0)  
  
}  
  
function draw() {  
  // very basic paint program  
  noStroke();  
  ellipse(mouseX,mouseY,20);  
}
```



BASIC SQUARES

```
function setup() {  
  createCanvas(windowWidth, windowHeight)  
  background (0)  
  
}
```

```
function draw() {  
  fill(255,255,255)  
  stroke(0,255,0)  
  strokeWeight(10)  
  square(400,200,100)  
  
  fill(100,100,255)  
  noStroke()  
  square(600, 200, 150, 10)  
  
  noFill()  
  stroke(255,100,255)  
  square(850, 200, 300)  
}
```



afternoon

- starting with the recreation of Vera Molnar's generative art made of squares
- introduction to the "for"-loop

- introduction of random function

SQUARE IN SQUARE

```
function setup() {
  createCanvas(windowWidth, windowHeight)
  background(255, 255, 245)
  rectMode(CENTER)
}

let dim = 200
let reduction = 30
let posX = 600
let posY = 300

function draw() {
  noFill()
  strokeWeight(1)
  stroke(0)
  square(posX, posY, dim)
  square(posX, posY, dim - (reduction * 1))
  square(posX, posY, dim - (reduction * 2))
  square(posX, posY, dim - (reduction * 3))
  square(posX, posY, dim - (reduction * 4))
  square(posX, posY, dim - (reduction * 5))

  // for-loop
  for(let i = 0; i < 6; i++) {
    square(posX-dim, posY, dim - (reduction * i))
  }

  for(let i = 0; i < 6; i++) {
    square(posX, posY+dim, dim - (reduction * i))
  }

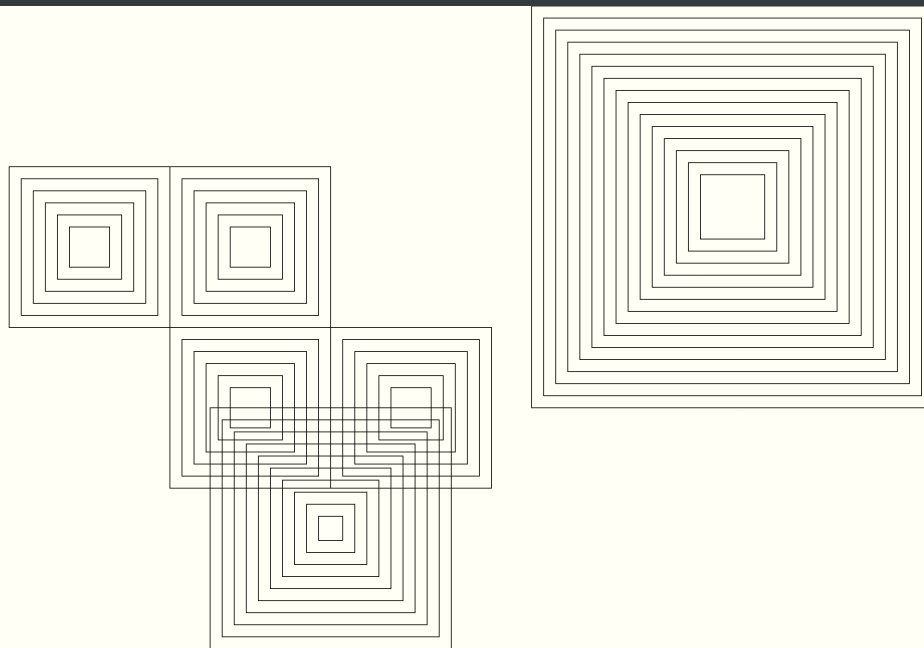
  for(let i = 0; i < 6; i++) {
    square(posX + dim, posY + dim, dim - (reduction * i))
  }
}
```

```

for(let i = 0; i < 10; i++) {
  square(posX + 100, posY+350, (dim + 100) - (reduction * i))
}

for(let i = 0; i < 15; i++) {
  square(posX + 600, posY-50, (dim + 300) - (reduction * i))
}

```



ROTATING SQUARE

```

let dimX = 300
let dimY = dimX
let num = 20
let reduction = dimX / num
let posX = 0
let posY = 0

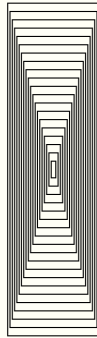
function setup() {
  createCanvas(windowWidth, windowHeight)
  background(255, 255, 245)
  rectMode(CENTER)
  angleMode(DEGREES)
}

```

```
posX = width / 2
posY = height / 2
}

function draw() {
  background(255, 255, 245)
  noFill()
  strokeWeight(1)
  stroke(0)
  dimX = (sin(frameCount * 4) * 300)
  num = 20
  reduction = dimX / num

  // for-loop
  for(let i = 0; i < num; i++) {
    rect(posX, posY,
      (dimX) - (dimX / num * i),
      (dimY) - (dimY / num * i)
    )
  }
}
```

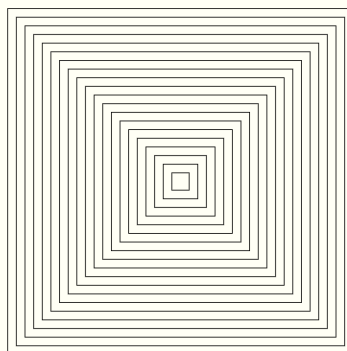


BASIC SIZE ANIMATION

```
let dimX = 300
let dimY = dimX
let num = 20
let reduction = dimX / num
let posX = 0
let posY = 0

function setup() {
  createCanvas(windowWidth, windowHeight)
  background(255, 255, 245)
  rectMode(CENTER)
  angleMode(DEGREES)
  posX = width / 2
  posY = height / 2
}
```

```
function draw() {  
  background(255, 255, 245)  
  noFill()  
  strokeWeight(1)  
  stroke(0)  
  //animation  
  //calculate the dimension of the square  
  //based on a sin function  
  dimX = 300 +(sin(frameCount * 4) * 150)  
  num = 20  
  reduction = dimX / num  
  
  // for-loop  
  for(let i = 0; i < num; i++) {  
    rect(posX, posY,  
      (dimX) - (dimX / num * i))  
  }  
}
```



RANDOM OFFSET ANIMATION

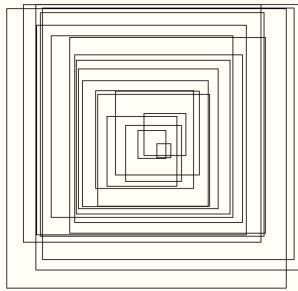
```
let dimX = 300
let dimY = dimX
let num = 20
let reduction = dimX / num
let posX = 0
let posY = 0

function setup() {
  createCanvas(windowWidth, windowHeight)
  background(255, 255, 245)
  rectMode(CENTER)
  angleMode(DEGREES)
  posX = width / 2
  posY = height / 2
}

function draw() {
  background(255, 255, 245)
  noFill()
  strokeWeight(1)
  stroke(0)
  //animation
  //calculate the dimension of the square
  //based on a sin function
  dimX = 200 +(sin(frameCount * 4) * 150)
  num = 20
  reduction = dimX / num

  // for-loop
  for(let i = 0; i < num; i++) {
    let offsetX = random(40)
    let offsetY = random(40)
    square(
      posX + offsetX,
      posY + offsetY,
```

```
(dimX) - (dimX / num * i))  
}  
}
```



WEDNESDAY

morning

- proceeding with the recreation of Vera Molnar's artwork
- introduction to the "if"-function
- how to create your own functions

COLOURED WIGGLING SQUARES

```
let dimX = 300  
let dimY = dimX  
let num = 20
```

```
let reduction = dimX / num
let posX = 0
let posY = 0
let x = 0
let y = 0

function setup() {
  createCanvas(windowWidth, windowHeight)
  background(255, 255, 245)
  rectMode(CENTER)
  angleMode(DEGREES)
  posX = width / 2
  posY = height / 2
}

function draw() {
  background(255, 255, 245)
  noFill()
  stroke(0)

  // for-loop (this draws several multicolour
  // squares wiggling within each other)
  //global animation
  //calculate the dimension of the square
  //based on a sin function
  //size dimX changes from 300 +1 to -1 (sin)
  dimX = 300 + sin(frameCount * 4) * 10
  num = 10
  reduction = dimX / num

  for(let i = 0; i < num; i++) {
    let offsetX = random(7)
    let offsetY = random(7)
    stroke(255, 0, 255)
    strokeWeight(3)
    if(i < 5) {
      strokeWeight(2)
```

```

        stroke(255, 255, 0)
    }
    if(i == 2) {
        stroke(0, 255, 255)
        strokeWeight(4)
    }
    if(i == 8) {
        stroke(0, 255, 0)
    }

    square(
        posX + offsetX,
        posY + offsetY,
        (dimX) - (reduction * i)
    )

    // here we implement function tmcs on the
    // position posX = 100 and posY = 100

}
tmcs(1200, 400, 250, 5, 7)
}

// this function draws swaures at position
// x and y, and the ymove randomly
// a tiny bit

function tmcs(x, y, dim, speed, num) {

    let dimension = dim + sin(frameCount * speed) * 10
    let reduction = dimension / num
    // adding let in front of a variable, the
    // variable is (re)defined. the definition
    // of the variable is only relevant for
    // this function as the definition is
    // within the function (local to function)
    // changing the function will not affect
    // anything outside the function

    for(let i = 0; i < num; i++) {

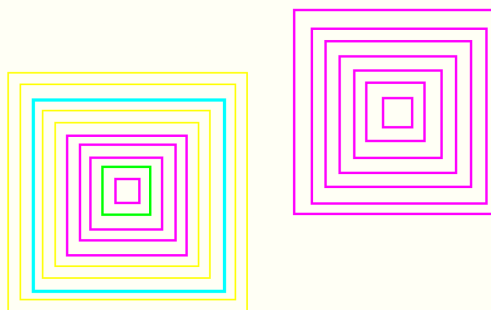
```

```

let offsetX = random(7)
let offsetY = random(7)
stroke(255, 0, 255)
strokeWeight(3)

square(
  x + offsetX,
  y + offsetY,
  (dimension) - (reduction * i)
)
}
}

```



MOLNAR GRID NOISE ANIMATION

```

let dimX = 300
let dimY = dimX

```

```

let num = 20
let reduction = dimX / num
let posX = 0
let posY = 0
let x = 0
let y = 0

function setup() {
  createCanvas(windowWidth, windowHeight)
  background(0)
  rectMode(CENTER)
  angleMode(DEGREES)
  posX = width / 2
  posY = height / 2
}

function draw() {
  background(0)
  noFill()
  stroke(0)
  let numX = 10

  for(let i = 0; i < numX; i++) {
    let dimension = width / numX
    let posX = (dimension / 2) + (i * dimension)
    for(let j = 0; j < 5; j++) {
      let posY = dimension / 2 + (j * dimension)
      tmcs(posX, posY, dimension, 10, 4)
    }
  }
  //noLoop()
}

// this function draws swaures at position
// x and y, and the ymove randomly
// a tiny bit

```

```

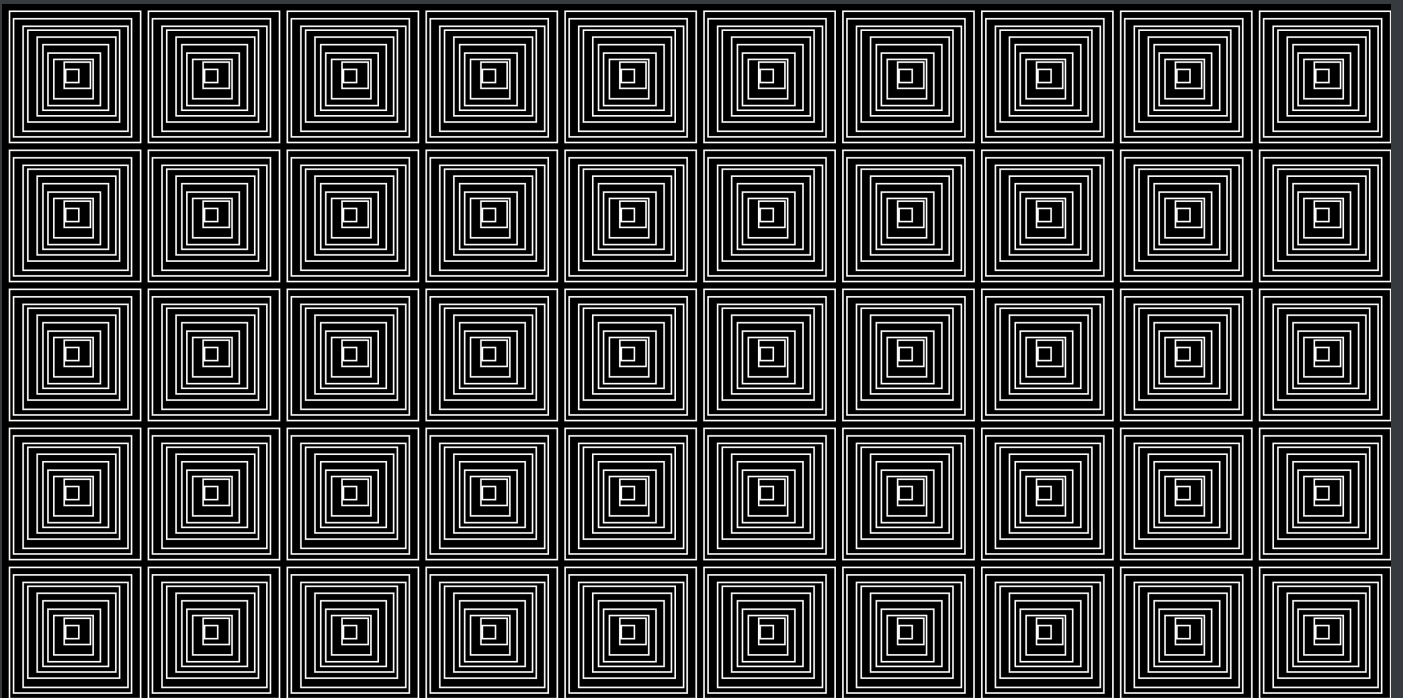
function tmcs(x, y, dim, num, speed) {

  let dimension = dim + sin(frameCount * speed) * 10
  let reduction = dimension / num

  for(let i = 0; i < num; i++) {
    let offsetX = noise (frameCount * 0.001, frameCount * 0.003, i) * 10
    let offsetY = noise (frameCount * 0.01, frameCount * 0.03, i) * 10
    stroke(255,255,255)
    strokeWeight(2)

    square(
      x + offsetX,
      y + offsetY,
      (dimension) - (reduction * i)
    )
  }
}

```



RANDOM COLOUR MOLNAR GRID

```
let dimX = 300
let dimY = dimX
let num = 20
let reduction = dimX / num
let posX = 0
let posY = 0
let x = 0
let y = 0

function setup() {
  createCanvas(windowWidth, windowHeight)
  background(0)
  rectMode(CENTER)
  angleMode(DEGREES)
  posX = width / 2
  posY = height / 2
}

function draw() {
  background(0)
  noFill()
  stroke(0)
  let numX = 10

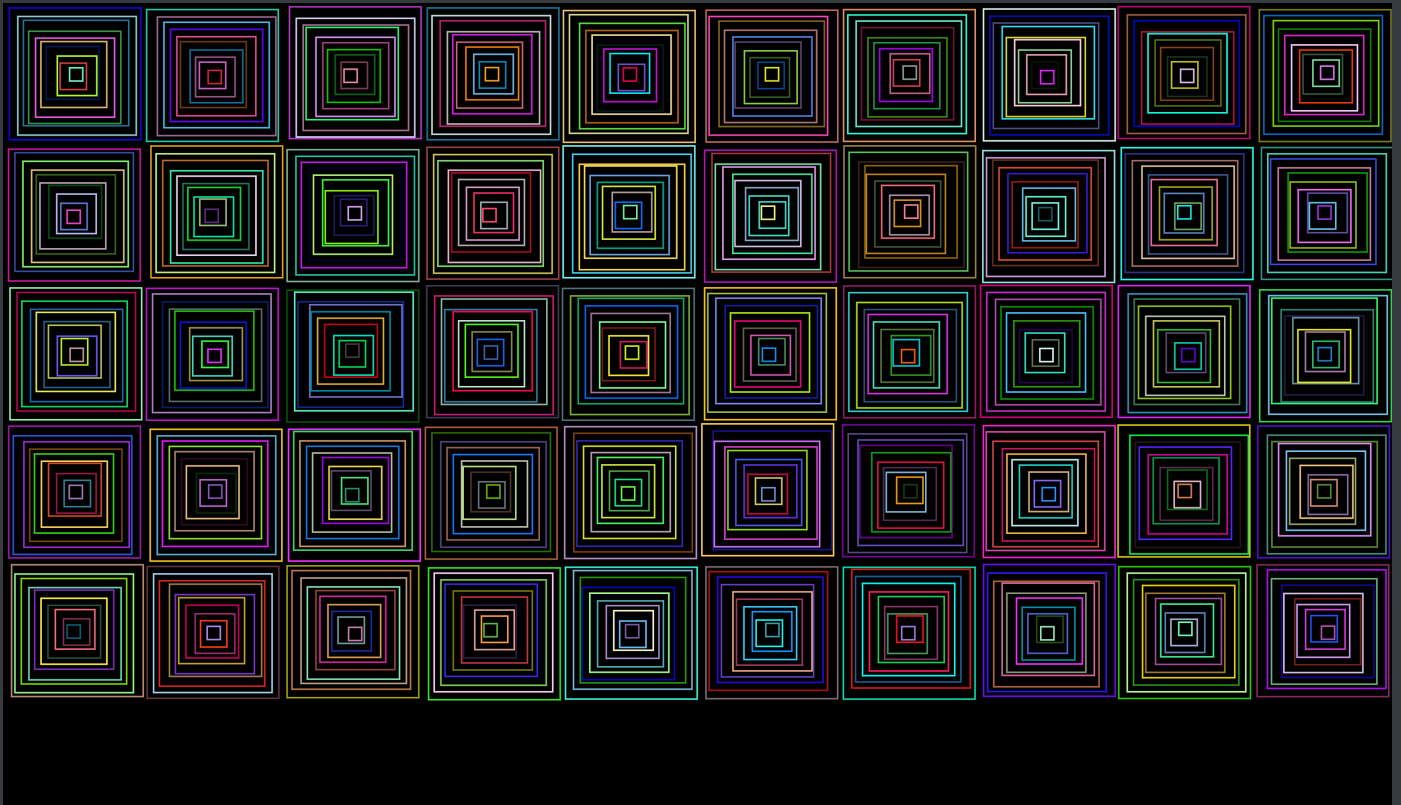
  for(let i = 0; i < numX; i++) {
    let dimension = width / numX
    let posX = (dimension / 2) + (i * dimension)
    for(let j = 0; j < 5; j++) {
      let posY = dimension / 2 + (j * dimension)
      tmcs(posX, posY, dimension, 10, 4)
    }
  }
}

// this function draws swaures at position
// x and y, and the ymove randomly
```



```
// a tiny bit
```

```
function tmcs(x, y, dim, num, speed) {  
  
  let dimension = dim + sin(frameCount * speed) * 10  
  let reduction = dimension / num  
  
  for(let i = 0; i < num; i++) {  
    let offsetX = random(7)  
    let offsetY = random(7)  
    let r = random (255)  
    let g = random (255)  
    let b = random (255)  
    stroke(r, g, b)  
    strokeWeight(2)  
  
    square(  
      x + offsetX,  
      y + offsetY,  
      (dimension) - (reduction * i)  
    )  
  }  
}
```



afternoon

- introduction to 3d space in p5.live
- recreation of DVD screensaver in 3D

BASIC 3D ANIMATION

```
let posX = 0
let posY=0
let boxDim = 100
function setup() {
  createCanvas(windowWidth, windowHeight, WEBGL)
  background(0)

}

function draw() {
  background(0)
```

```

orbitControl()
fill(255)
stroke(0,255,0)

//posX++ or posX+=1 or posX = posX+1 (to the right)
//posX-- or posX = posX-1 (to the left)

posX+=10
posY = posY - 10

if (posX > (width/2)+(boxDim/2)) {
  posX = 0 - (width/2)
}

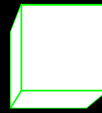
if (posY < - (height/2)) {
  posY = 0
}

push()
translate (posX,posY,0)
box(boxDim)
pop()

// coordinate system
strokeWeight(2)
push()
stroke(255, 0, 0)
line(0,0,0, width, 0, 0)
stroke(0, 255, 0)
line(0,0,0, 0, height, 0)
stroke(0, 0, 255)
line(0,0,0, 0, 0, 1000)
pop()

```

```
}
```



3D ANIMATION INSPIRED BY DVD-SCREENSAVER

```
let x;  
let y;  
let xspeed;  
let yspeed;  
let zspeed;  
let depth = 2000  
let r, g, b;  
  
function setup() {  
  createCanvas(windowWidth, windowHeight, WEBGL);  
  //initialise the variables for the  
  //position of the cube at the  
  //beginning!  
  x = random(width);  
  y = random(height);
```

```
z = random (-depth, 0);

//initialises speed at which the
//cube moves in each direction!
xspeed = 4;
yspeed = 4;
zspeed = 4;

pickcolor();

}

function pickcolor() {
  r = random(255);
  g = random(255);
  b = random(255);
}

function draw() {
  //set origin of coordinate system
  //to top left corner of canvas!
  translate(-(width / 2), -height / 2, 0)
  background(0);
  fill(r, g, b);
  stroke(0)
  //rect(x, y, 20);
  //image(dvd, x, y, 80, 60);

  push()
  translate(x, y, z) // moves box to new pos
  box(100)
  pop()

  x = x + xspeed;
  y = y + yspeed;
  z = z + zspeed;
```

```
if(z >= 0) {
    zspeed = -zspeed;
    z = 0;
    pickcolor();
} else if(z <= -depth) {
    zspeed = -zspeed;
    z = -depth;
    pickcolor();
}
```

```
if(x >= width) {
    xspeed = -xspeed;
    x = width;
    pickcolor();
} else if(x <= 0) {
    xspeed = -xspeed;
    x = 0;
    pickcolor();
}
```

```
if(y >= height) {
    yspeed = -yspeed;
    y = height;
    pickcolor();
} else if(y <= 0) {
    yspeed = -yspeed;
    y = 0;
    pickcolor();
}
```

```
}
```

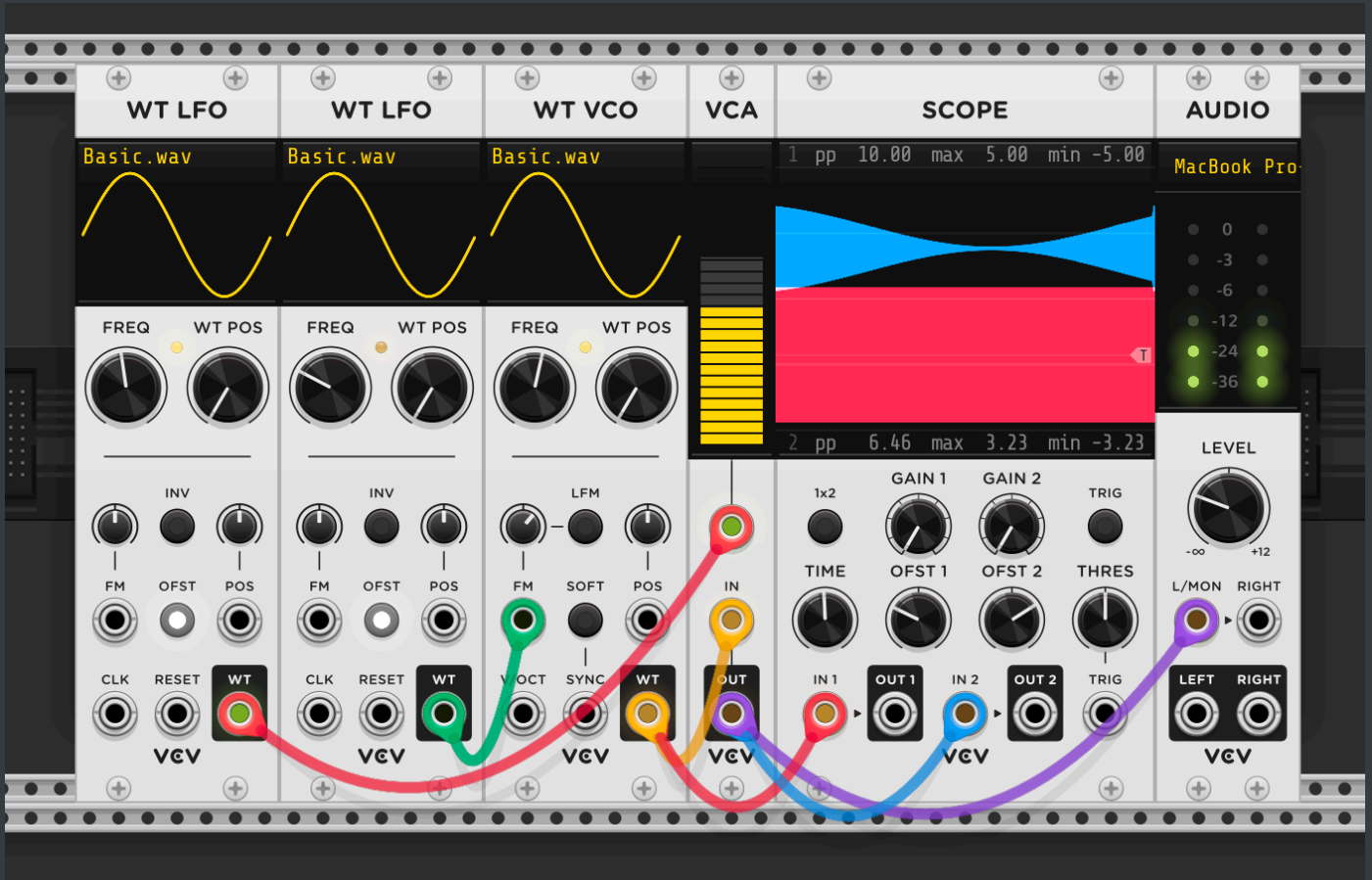


THURSDAY

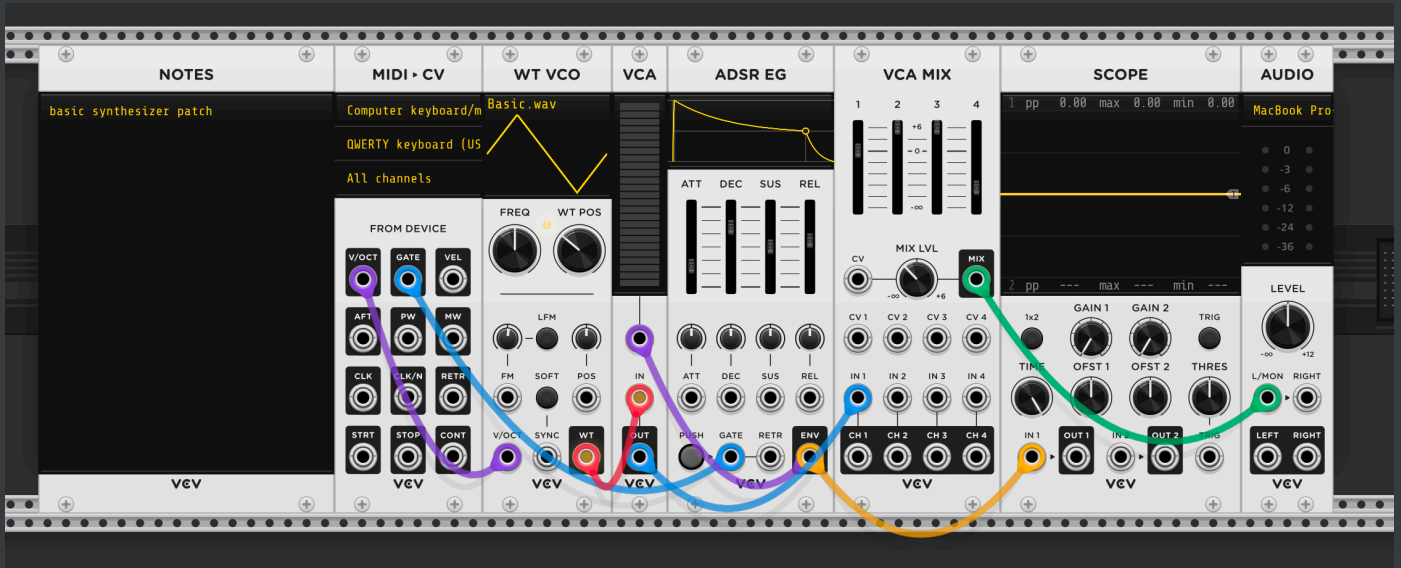
morning

- brief introduction to sound creating programmes: Strudel, VSV Rack
- introduction to VCV Rack 2

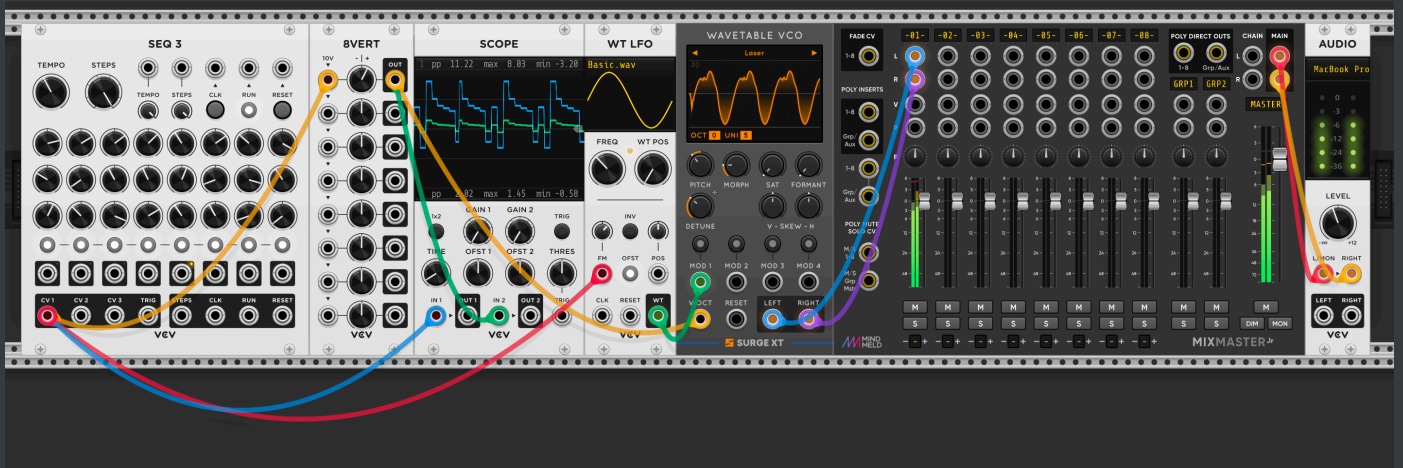
BASIC SIREN-LIKE SOUND



BASIC MELODY



KEYBOARD SOUND SYNTHESIZER



afternoon

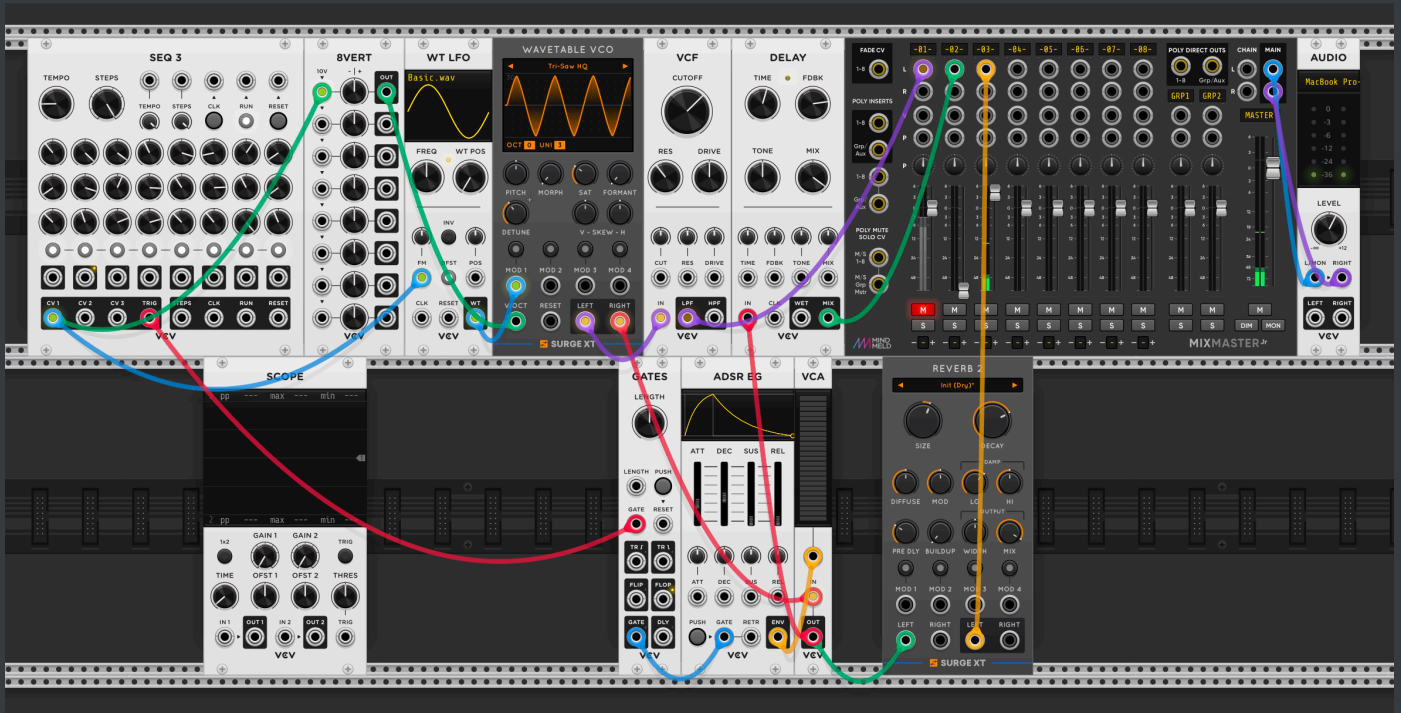
absent due to introduction to the 3D printing workshop

FRIDAY

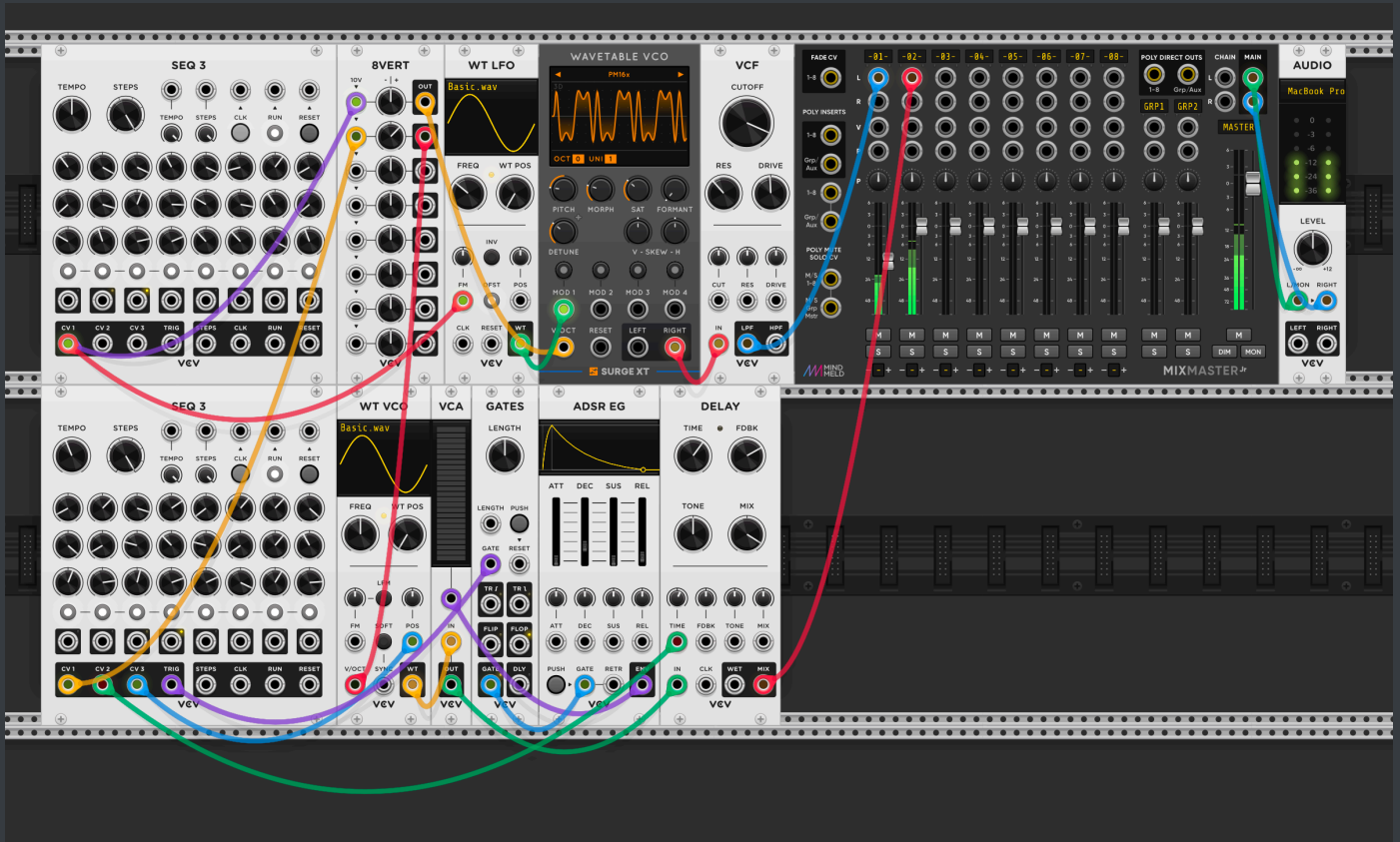
morning

- collect inspiration for generative art / live coding
- call with Yann:
 - general clarification of end product / goal and mid term presentation
 - look at inspiration -> distortion of typography using Hydra
- catching up on what I've missed on Thursday

SAMPLE 004



SAMPLE 005



afternoon

- experiment with text in p5.live
- watching tutorials on p5 / vcv rack

IMPLEMENTING TYPOGRAPHY IN P5 LIVE

```
function setup() {  
  createCanvas(windowWidth, windowHeight)  
  
}  
  
function draw() {  
  background(255, 255, 255, 2)  
  textFont('Courier', 200)  
  text('hallo', mouseX, mouseY)  
}
```

